

How We Build the Persistence WAL — Structural Splitting, Code & Data Walkthrough

For products with no maturity date at all — call accounts, current accounts, notice deposits — but where the balance clearly has real staying power, we don't assign the whole account one single WAL. We split the balance into behavioral tranches instead, using the Minimum Balance Method. See `docs/behavioural_wal.md` for why this product type needs its own treatment; here's the split itself, as we implement it in `expand_structural_split_tranches`.

The split

Every `STRUCTURAL_SPLIT`-type account expands into **up to four tranche rows** in our pipeline — `OVERNIGHT`, `SHORT TERM`, `MEDIUM TERM`, `LONG TERM` — each carrying its own share of the balance and its own WAL:

1. **Tranche balance.** We compute each tranche's `TRANCHE_OUT_BAL` as the account's real `OUT_BAL` multiplied by that tranche's percentage from our structural-split reference sheet, keyed by (branch, ALCO class). We deliberately leave the account's real, full `OUT_BAL` untouched on every tranche row — only `TRANCHE_OUT_BAL` varies. Overwriting `OUT_BAL` itself with a split portion would make an unpopulated tranche indistinguishable from a genuinely zero-balance account at a glance, and we learned that the hard way.
2. **Tranche WAL.** Each tranche's `CALCULATED_WAL` comes from a separate reference sheet, keyed on **(branch, tenor classification, type)** — a three-key match, not two. We need the `type` key (asset vs. liability) because the same (branch, tenor class) combination can carry a genuinely different final WAL depending on which side of the balance sheet it's pricing. Type itself isn't a column that exists yet at this point in our cascade — we derive it on the fly from the account's GL line, using the same asset-starts-with-1 convention we use elsewhere in the pipeline.
3. **Non-split products pass through unchanged** — `TRANCHE_OUT_BAL` always exists on our output regardless of WAL type, set equal to the account's own `OUT_BAL` for contractual/prepayment rows, so anything downstream always has one column to read for "this row's own share of balance" without needing to branch on whether a split happened.
4. **We keep zero-valued tranches, we don't drop them** — a 0% split, or a reference-data gap, still produces a visible row rather than disappearing silently. That's useful for us in

tracking exactly where reference-data coverage is incomplete.

Example

One account, four tranche rows:

ACC_NO	TENOR_CLASSIFICATION	TRANCHE_OUT_BAL	CALCULATED_WAL
CALL001	OVERNIGHT	90,000.00	0.0027
CALL001	SHORT TERM	110,000.00	0.62
CALL001	MEDIUM TERM	175,000.00	1.85
CALL001	LONG TERM	125,000.00	4.20

This is a genuinely different shape of answer than a single-WAL account gives — not “this account has WAL X,” but “this account is priced as four separate pieces, each with its own WAL.” That’s exactly what lets us price the volatile and persistent portions of the same balance differently, instead of picking one figure that’s wrong for most of it either way.

Where we hit real limits

- **Reference-sheet coverage drives everything.** A (branch, ALCO class) combination with no row in our structural-split sheet, or a (branch, tenor class, type) combination with no row in our tenor-class-WAL sheet, resolves to a 0%-split or missing-WAL tranche rather than an error — deliberately visible so we can track the gaps, but it does mean split coverage needs active maintenance as we add new products or branches.
- **Behavioral, not contractual — inherently an estimate.** Unlike contractual WAL, every number here reflects observed balance persistence rather than a contractual fact. We revisit it periodically against actual observed behavior; we don’t treat it as a fixed, one-time calibration.

See also `docs/behavioural_wal.md` and `docs/prepayment_wal.md`.